

Module 7: Lesson 3

Multilayer Perceptrons (MLP)



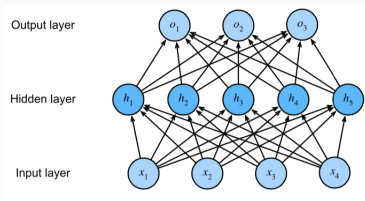
Outline

- ▶ Multilayer Perceptrons (MLP) and hidden layers
- ▶ Activation functions: ReLU, sigmoid, tanh
- ▶ Forward and Backward propagation
- ▶ Early-stopping

Multilayer perceptron (MLP)

Now that we know how to implement linear regression as a neural network in Tensorflow, let's introduce a truly deep model. We will do so using the simplest type of deep networks: **multilayer perceptron (MLP)**.

One of the main features of any deep network is a number of hidden layers. These layers, as we will see shortly, will allow us to explore non-linear relationships in the data:

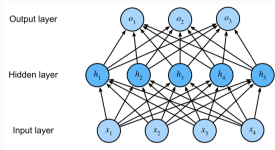


- How does data go from the input to output layer (O_q)?

$$H = XW^{(1)} + b^{(1)}$$

$$O = HW^{(2)} + b^{(2)}$$

Multilayer perceptron (MLP)



- What are the dimensions of each of these terms?
 - X is a matrix of $n \times d$ (a minibatch of n examples for d inputs)
 - $W^{(1)}$ is a matrix of $d \times h$ (d inputs for h units in the hidden layer). Here $h = 5$
 - $b^{(1)}$ is a vector of $1 \times h$ (1 bias term for each unit in hidden layer)
 - H is a matrix of $(n \times d)(d \times h) + (1 \times h) = n \times h$ (a minibatch of n examples for h hidden-layer units)
 - $W^{(2)}$ is a matrix of $h \times q$ (h units in the hidden layer by q units in the output layer). Here $q = 3$
 - $b^{(2)}$ is a vector of $1 \times q$ (1 bias term for each unit in output layer)
 - O is a matrix of $(n \times h)(h \times q) + (1 \times q) = n \times q$

Activation functions

Now, if all these layers are fully connected and always active, there is little actual gain from the linear regression case → **where is the non-linearity?**

Our secret ingredient is going to be a **non-linear activation function**. These functions transform inputs, in a non-linear way, to decide whether neurons of a layer are activated (or not).

- Thus, for an activation function $\sigma()$, our previous set of equations will look like:

$$H = \sigma \left(XW^{(1)} + b^{(1)} \right)$$

$$O = HW^{(2)} + b^{(2)}$$

The previous example had 1 hidden layer. If more, we could apply a (different) activation function to each one:

$$H^{(1)} = \sigma_1 \left(XW^{(1)} + b^{(1)} \right)$$

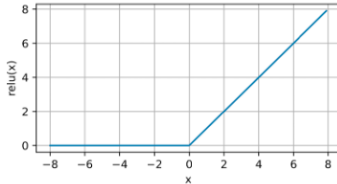
$$H^{(2)} = \sigma_2 \left(H^{(1)} W^{(2)} + b^{(2)} \right)$$

$$O = H^{(2)} W^{(3)} + b^{(3)}$$

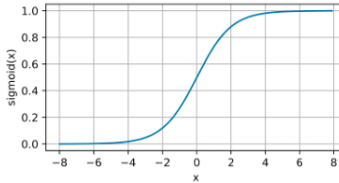
Activation functions

Since it is actually the activation function that ultimately brings non-linearity to our deep networks, it seems reasonable to review the main one used in practice:

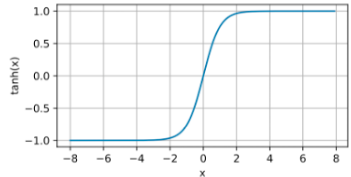
$$\text{ReLU}(x) = \max(x, 0).$$



$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$



$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$



Forward and Backward propagation

There are 2 key algorithms involved in the training of our neural networks: **forward and backward propagation**.

► **Forward propagation (or forward pass):**

- Refers to the calculation (and storage) of the intermediate variables (and outputs) of the network, in layer-to-layer order, from input to output layers.

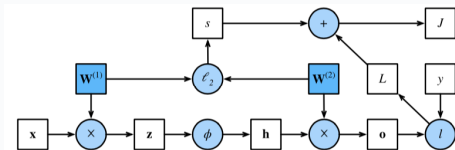


Fig. 5.3.1: Computational graph of forward propagation.

where, for a one hidden-layer network and bias term $b = 0$:

$z = W^{(1)}x \rightarrow h = \phi(z) \rightarrow o = W^{(2)}h \rightarrow$ and J is the regularized-cost function using an ℓ_2 norm.

Forward and Backward propagation

Forward prop is highly interconnected with **backward propagation**:

► **Backward propagation (backpropagation):**

- Refers to the method of calculating gradient of NN parameters, and updating them.
- The method essentially consists of calculating the partial derivative of the cost function with respect to the different parameters (i.e., $W^{(l)}$).
- To achieve this, it relies on the chain rule:

$$\frac{\partial Z}{\partial X} = \text{prod} \left(\frac{\partial Z}{\partial Y}, \frac{\partial Y}{\partial X} \right)$$

- The backprop algorithm will start with the gradient of the loss function and end obtaining the gradient of the model weights closest to the input layer (i.e., $W^{(1)}$):

$$\frac{\partial J}{\partial W^{(1)}} = \text{prod} \left(\frac{\partial J}{\partial z}, \frac{\partial z}{\partial W^{(1)}} \right) + \text{prod} \left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial W^{(1)}} \right) = \frac{\partial J}{\partial z} x^T + \lambda W^{(1)}$$

Early stopping

The forward prop and backprop algorithms will effectively train the model using iterative process:

- ▶ When do iterations/training stop?

The easy answer to this question is **when the stopping criteria is met**. But this begs another question:

- ▶ What is the stopping criteria?

There are many possible choices. For now, we are going to see one of the options: **Early Stopping**.

- ▶ Early stopping serves as both a stopping criterion and a regularization technique.
- ▶ Essentially, it consists of constraining the number of epochs of training. **How?**
 - During training, monitor the error in validation set (usually checking after each epoch).
 - **Stopping criteria:** stop when the validation error has not decreased by more than some amount ϵ for some number of epochs
 - How many is some epochs? → **patience**
 - Importantly, the **validation set** must NOT be a part of training nor test sets!

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ Multilayer Perceptron (MLP) and hidden layers
- ▶ Activation functions: ReLU, sigmoid, tanh
- ▶ Forward and Backward propagation
- ▶ Early-stopping

⇒ **References for this lesson:**

Chapter 5, Sections 5.1, 5.2, and 5.3 of [Zhang, Aston, et al. *Dive into Deep Learning*. 2021.](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement our first deep network (MLP) using Tensorflow for our trading strategy aimed at timing the momentum factor.